

CASA0007_project_technical_annex

August 26, 2026

```
[1]: # pip install jenkspy
```

1 High-Level Overview of Direct Employment in London's Data Centre Sector: Technical Annex

1.1 Overview

link to repo: https://github.com/a-saveleva/casa0007_project

Recently, data centres (DC) were designated as Critical National Infrastructure in the UK, placing them alongside essential services such as water, energy, and emergency response. According to one of our sources (an architect involved in data centre planning and design), the UK currently has around 500 data centres, and this number is expected to increase by approximately 20% over the next 5–10 years.

With a substantial number of DCs in the development pipeline, questions are increasingly being raised about the economic benefits these facilities bring to local areas. This document presents a step-by-step regression modelling and clustering exercise that addresses the following questions:

1. How many data centres are there in London?
2. What are their basic characteristics, particularly those relevant to planning (e.g. size, employment, and energy use)?
3. How many people are employed in data centres, and how does this relationship change with DC size, location, or other characteristics?

```
[2]: # pip install -U kaleido
```

```
[10]: import pandas as pd
import geopandas as gpd
import numpy as np
import matplotlib.pyplot as plt
import math
import patsy

from scipy.stats import ks_2samp
from scipy.stats import norm

import plotly.graph_objects as go
import plotly.express as px
```

```

from plotly.subplots import make_subplots
import seaborn as sns
import kaleido
kaleido.get_chrome_sync()

import statsmodels
import statsmodels.api as sm
from statsmodels.tools.tools import maybe_unwrap_results
from statsmodels.graphics.gofplots import ProbPlot
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.formula.api as smf
from sklearn.neighbors import NearestNeighbors
from typing import Type

# from sklearn.impute import KNNImputer
# from sklearn.preprocessing import StandardScaler
# from sklearn.metrics import silhouette_score, silhouette_samples
# from sklearn.cluster import KMeans
from sklearn.cluster import DBSCAN

pd.set_option('display.max_columns', None)

```

1.2 Loading and EDA

EDA textual outputs are presented in the main report.

```

[4]: #
      ↪ -----
      # Loading the previously validated dataset. See more in data_load folder of the ↪
      ↪ repository
      #
      ↪ -----
data_ldn = pd.read_csv(
    "data_load/final_output.csv", na_values=["NULL", "null", "None", "", "[]", ↪
    ↪ "NaN"])
data_ldn["geometry"] = gpd.GeoSeries.from_wkt(data_ldn["geometry_point"])
data_ldn = gpd.GeoDataFrame(data_ldn, geometry="geometry", crs="EPSG:4326")
#
      ↪ -----
      # For plotting
      #
      ↪ -----
gla_boundary = gpd.read_file("data_load/gla/London_GLA_Boundary.shp").
    ↪ to_crs(4326)
#
      ↪ -----
      # Filtering out rows that are not of interest at the moment

```

```

#
↳
data_ldn_exstg = data_ldn[data_ldn["completed"] == "Completed"].drop(axis=1,
↳columns=['completed']) #taking only the data centres that were built
data_ldn_exstg = data_ldn_exstg[data_ldn_exstg["lpa_name"] != "Slough"]
data_ldn_exstg = data_ldn_exstg.sort_values(by="data_centre_since",
↳ascending=True)
data_ldn_exstg.head(1)
#
↳
# Creating a set of new columns for testing
#
↳
data_ldn_exstg["b8_gia_sqm_per_ha"] = data_ldn_exstg["b8_gia_sqm"] /
↳data_ldn_exstg["application_details_site_area"]
data_ldn_exstg["b8_gia_ha_per_site_ha"] = (data_ldn_exstg["b8_gia_sqm"] /10000)
↳/ data_ldn_exstg["application_details_site_area"]
data_ldn_exstg["b8_gia_1000_sqm"] = data_ldn_exstg["b8_gia_sqm"] / 1000
data_ldn_exstg["b8_total_staff_per_1000_sqm"] =
↳data_ldn_exstg["b8_total_staff_fte"] / data_ldn_exstg["b8_gia_1000_sqm"]
# #scaling years column to keep condition n at bay
# col = "data_centre_since"
# min_year = data_ldn_exstg[col].min()
# max_year = data_ldn_exstg[col].max()
# year_range = max_year - min_year
# data_ldn_exstg["data_centre_since_scaled"] = ((data_ldn_exstg[col] -
↳min_year) + 1)

```

1.2.1 Column definitions

Status and ownership - Unique PA reference - **id** - DC postcode - **postcode** - DC address - **address** - Category col indicating the state of completion - **completed** - Date when DC was operational for the first time in this building. Note the size and staff numbers reflect the current situation, not when DC opened - **data_centre_since** - Year/date when the first pre-app stage document was received - **preapp_started** - Boolean col indicating whether DC is enterprise or any other type - **enterprise** - Current owner/occupier of DC - **current_occupier** - Company website - **website** - Unique property reference - **uprn** - Local authority (borough) name - **lpa_name**. Note a few of DC in Slough were manually added as the information became available. Slough DC will only be used for size and staff analysis, not for the spatial clustering

Attached planning application - PA code (contains repetitions with unique other columns, use id for cross-comparing) - **lpa_app_no** - PA webpage - **url_planning_app** - PA decision date - **decision_date**. Note it does not always represent a moment when the DC was created - PA description. Note it sometimes refers to PAs with additions to DC, not its development - **description** - Notes about the application and current use - **notes** - PA system update - **last_updated** - Helper

column with the data source – **is_full_planning**

Spatial parameters - Gross internal area of the DC use - **b8_gia_sqm** - Site area in hectares – **application_details_site_area** - Number of storeys in the DC building, or stories occupied by the DC if other uses present - **b8_no_storeys** - Height of the DC in meters **b8_max_height_m** - List of other uses within the building with the DC – **other_use_classes_present_within_b8_building** - Geometry cols - **geometry_point**, **geometry** -

Technical parameters - Total facility energy - **utility_total_electr_capacity_mva** - IT equipment energy, available for customers – **it_load_capacity_mw** - Data center power density i - **density_kw_max_per_sqm** - Tier benchmark – **tier** - Power Usage Effectiveness: ratio of total facility energy to IT equipment energy- **pue**

Employment data - Total staff number in the DC - **b8_total_staff_fte** - N of security staff - **security_staff** - N of engineering/technical staff - **site_engineers_staff** - N of administrative staff - **managers_staff** - N of cleaning/housekeeping staff - **cleaners_staff** - N of visitor staff, typically client technicians - **visitor_staff_not_on_site** - N of staff working within one cluster, if more than one DC is in the area. Mostly refers to admin/technicians - **cluster_staff_not_on_site** - N of staff present on site per day - **b8_employees_at_a_day**

```
[5]: data_ldn_extstg.describe()
```

```
[5]:      application_details_site_area  data_centre_since  b8_no_storeys  \
count      40.000000      47.000000      39.000000
mean        1.290802      2009.106383      4.051282
std         1.422233        9.132441      3.276300
min         0.009000      1979.000000      1.000000
25%         0.211250      2001.500000      2.000000
50%         1.005000      2010.000000      2.000000
75%         1.980000      2017.000000      6.000000
max         7.540000      2025.000000     13.000000
```

```
      b8_max_height_m  b8_gia_sqm  b8_total_staff_fte  \
count      29.000000      45.00000      25.000000
mean       615.824931     13281.16744      76.440000
std       3208.869823     14794.65748      77.907039
min         4.400000       24.00000       5.000000
25%        10.000000      2500.00000      20.000000
50%        12.500000      8450.00000      45.000000
75%        29.753000     18354.88000      92.000000
max       17300.00000     69829.00000     262.000000
```

```
      utility_total_electr_capacity_mva  it_load_capacity_mw  \
count      17.000000      27.000000
mean       29.247059      14.075926
std        22.302610      13.710295
```

min	2.000000	1.350000
25%	11.700000	4.000000
50%	24.000000	10.000000
75%	46.000000	19.500000
max	80.000000	64.000000

	density_kw_max_per_sqm	pue	tier	security_staff	\
count	27.000000	14.000000	19.0	5.000000	
mean	1.693333	1.322071	3.0	9.800000	
std	0.680390	0.293528	0.0	4.024922	
min	0.250000	0.700000	3.0	5.000000	
25%	1.225000	1.250000	3.0	6.000000	
50%	1.430000	1.300000	3.0	12.000000	
75%	2.345000	1.300000	3.0	12.000000	
max	3.000000	2.100000	3.0	14.000000	

	cleaners_staff	managers_staff	site_engineers_staff	\
count	3.000000	4.000000	4.000000	
mean	4.333333	9.750000	25.250000	
std	2.886751	13.524669	27.801379	
min	1.000000	2.000000	7.000000	
25%	3.500000	2.750000	7.750000	
50%	6.000000	3.500000	14.000000	
75%	6.000000	10.500000	31.500000	
max	6.000000	30.000000	66.000000	

	visitor_staff_not_on_site	cluster_staff_not_on_site	\
count	6.000000	1.0	
mean	10.500000	13.0	
std	14.570518	NaN	
min	1.000000	13.0	
25%	5.000000	13.0	
50%	5.500000	13.0	
75%	6.000000	13.0	
max	40.000000	13.0	

	b8_employees_at_a_day_reported	total/day ratio	\
count	11.000000	4.000000	
mean	25.636364	2.990000	
std	18.704399	1.134578	
min	7.000000	1.970000	
25%	17.500000	2.307500	
50%	20.000000	2.710000	
75%	25.500000	3.392500	
max	62.000000	4.570000	

b8_employees_at_a_day_estimate	b8_gia_sqm_per_ha	\
--------------------------------	-------------------	---

count	11.000000	38.000000
mean	25.636364	22675.986831
std	18.704399	30500.204891
min	7.000000	155.844156
25%	17.500000	6146.147847
50%	20.000000	12881.960828
75%	25.500000	25593.487395
max	62.000000	136800.000000

	b8_gia_ha_per_site_ha	b8_gia_1000_sqm	b8_total_staff_per_1000_sqm
count	38.000000	45.000000	25.000000
mean	2.267599	13.281167	13.607616
std	3.050020	14.794657	26.471411
min	0.015584	0.024000	0.560224
25%	0.614615	2.500000	2.663329
50%	1.288196	8.450000	4.142052
75%	2.559349	18.354880	15.789474
max	13.680000	69.829000	128.645782

```
[6]: #
# Printing medians to insert into the main text
#
medians = data_ldn_exstg.select_dtypes(include="number").median()
print(medians)
```

application_details_site_area	1.005000
data_centre_since	2010.000000
b8_no_storeys	2.000000
b8_max_height_m	12.500000
b8_gia_sqm	8450.000000
b8_total_staff_fte	45.000000
utility_total_electr_capacity_mva	24.000000
it_load_capacity_mw	10.000000
density_kw_max_per_sqm	1.430000
pue	1.300000
tier	3.000000
security_staff	12.000000
cleaners_staff	6.000000
managers_staff	3.500000
site_engineers_staff	14.000000
visitor_staff_not_on_site	5.500000
cluster_staff_not_on_site	13.000000
b8_employees_at_a_day_reported	20.000000
total/day ratio	2.710000
b8_employees_at_a_day_estimate	20.000000
b8_gia_sqm_per_ha	12881.960828

```

b8_gia_ha_per_site_ha          1.288196
b8_gia_1000_sqm                8.450000
b8_total_staff_per_1000_sqm    4.142052
dtype: float64

```

```

[7]: # -----
# Helper functions
# -----

def make_transparent(color, alpha=0.3):
    if color.startswith("rgb"):
        return color.replace(")", f", {alpha})").replace("rgb", "rgba")
    elif color.startswith("#"):
        r, g, b = tuple(int(color[i:i+2], 16) for i in (1, 3, 5))
        return f"rgba({r},{g},{b},{alpha})"
    return color

# https://www.statsmodels.org/dev/examples/notebooks/generated/
# ↳ linear_regression_diagnostics_plots.html
style_talk = "seaborn-talk" # refer to plt.style.available

class LinearRegDiagnostic:
    """
    Diagnostic plots to identify potential problems in a linear regression fit.
    Mainly,
        a. non-linearity of data
        b. Correlation of error terms
        c. non-constant variance
        d. outliers
        e. high-leverage points
        f. collinearity

    Authors:
        Prajwal Kafle (p33ajkafle@gmail.com, where 3 = r)
        Does not come with any sort of warranty.
        Please test the code one your end before using.

        Matt Spinelli (m3spinelli@gmail.com, where 3 = r)
        (1) Fixed incorrect annotation of the top most extreme residuals in
            the Residuals vs Fitted and, especially, the Normal Q-Q plots.
        (2) Changed Residuals vs Leverage plot to match closer the y-axis
            range shown in the equivalent plot in the R package ggfortify.
        (3) Added horizontal line at y=0 in Residuals vs Leverage plot to
            match the plots in R package ggfortify and base R.
        (4) Added option for placing a vertical guideline on the Residuals
            vs Leverage plot using the rule of thumb of  $h = 2p/n$  to denote
            high leverage (high_leverage_threshold=True).
    """

```

```

(5) Added two more ways to compute the Cook's Distance (D) threshold:
    * 'baseR':  $D > 1$  and  $D > 0.5$  (default)
    * 'convention':  $D > 4/n$ 
    * 'dof':  $D > 4 / (n - k - 1)$ 
(6) Fixed class name to conform to Pascal casing convention
(7) Fixed Residuals vs Leverage legend to work with loc='best'
"""

def __init__(
    self,
    results: Type[statsmodels.regression.linear_model.
↳RegressionResultsWrapper],
) -> None:
    """
    For a linear regression model, generates following diagnostic plots:

    a. residual
    b. qq
    c. scale location and
    d. leverage

    and a table

    e. vif

    Args:
        results (Type[statsmodels.regression.linear_model.
↳RegressionResultsWrapper]):
            must be instance of statsmodels.regression.linear_model object

    Raises:
        TypeError: if instance does not belong to above object

    Example:
    >>> import numpy as np
    >>> import pandas as pd
    >>> import statsmodels.formula.api as smf
    >>> x = np.linspace(-np.pi, np.pi, 100)
    >>> y = 3*x + 8 + np.random.normal(0,1, 100)
    >>> df = pd.DataFrame({'x':x, 'y':y})
    >>> res = smf.ols(formula="y ~ x", data=df).fit()
    >>> cls = Linear_Reg_Diagnostic(res)
    >>> cls(plot_context="seaborn-v0_8-paper")

    In case you do not need all plots you can also independently make an
↳individual plot/table
    in following ways

```



```

>>> cls = Linear_Reg_Diagnostic(res)
>>> cls.residual_plot()
>>> cls.qq_plot()
>>> cls.scale_location_plot()
>>> cls.leverage_plot()
>>> cls.vif_table()
"""

if (
    isinstance(
        results, statsmodels.regression.linear_model.
↳RegressionResultsWrapper
    )
    is False
):
    raise TypeError(
        "result must be instance of statsmodels.regression.linear_model.
↳RegressionResultsWrapper object"
    )

self.results = maybe_unwrap_results(results)

self.y_true = self.results.model.endog
self.y_predict = self.results.fittedvalues
self.xvar = self.results.model.exog
self.xvar_names = self.results.model.exog_names

self.residual = np.array(self.results.resid)
influence = self.results.get_influence()
self.residual_norm = influence.resid_studentized_internal
self.leverage = influence.hat_matrix_diag
self.cooks_distance = influence.cooks_distance[0]
self.nparams = len(self.results.params)
self.nresids = len(self.residual_norm)

def __call__(self, plot_context="seaborn-v0_8-paper", **kwargs):
    # print(plt.style.available)
    # GH#9157
    if plot_context not in plt.style.available:
        plot_context = "default"
    with plt.style.context(plot_context):
        fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(10, 10))
        self.residual_plot(ax=ax[0, 0])
        self.qq_plot(ax=ax[0, 1])
        self.scale_location_plot(ax=ax[1, 0])
        self.leverage_plot(

```

```

        ax=ax[1, 1],
        high_leverage_threshold=kwargs.get("high_leverage_threshold"),
        cooks_threshold=kwargs.get("cooks_threshold"),
    )
    plt.show()

    return (
        self.vif_table(),
        fig,
        ax,
    )

def residual_plot(self, ax=None):
    """
    Residual vs Fitted Plot

    Graphical tool to identify non-linearity.
    (Roughly) Horizontal red line is an indicator that the residual has a
    ↪ linear pattern
    """
    if ax is None:
        fig, ax = plt.subplots()

    sns.residplot(
        x=self.y_predict,
        y=self.residual,
        lowess=True,
        scatter_kws={"alpha": 0.5},
        line_kws={"color": "red", "lw": 1, "alpha": 0.8},
        ax=ax,
    )
    # annotations
    residual_abs = np.abs(self.residual)
    abs_resid = np.flip(np.argsort(residual_abs), 0)
    abs_resid_top_3 = abs_resid[:3]

    for i in abs_resid_top_3:
        ax.annotate(i, xy=(self.y_predict[i], self.residual[i]), color="C3")
        # ax.annotate(residual_abs, xy=(self.y_predict[i], self.
    ↪ residual[i]), color="C5")

    # for i in abs_resid_top_3:
    #     idx_label = self.residual.index[i] # <-- actual df index label
    #     ax.annotate(idx_label, xy=(self.y_predict[i], self.residual[i]),
    ↪ color="C3")

    ax.set_title("Residuals vs Fitted", fontweight="bold")

```

```

ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
return ax

def qq_plot(self, ax=None):
    """
    Standarized Residual vs Theoretical Quantile plot

    Used to visually check if residuals are normally distributed.
    Points spread along the diagonal line will suggest so.
    """
    if ax is None:
        fig, ax = plt.subplots()

    QQ = ProbPlot(self.residual_norm)
    fig = QQ.qqplot(line="45", alpha=0.5, lw=1, ax=ax)

    # annotations
    abs_norm_resid = np.flip(np.argsort(np.abs(self.residual_norm)), 0)
    abs_norm_resid_top_3 = abs_norm_resid[:3]
    for i, x, y in self.__qq_top_resid(
        QQ.theoretical_quantiles, abs_norm_resid_top_3
    ):
        ax.annotate(i, xy=(x, y), ha="right", color="C3")

    ax.set_title("Normal Q-Q", fontweight="bold")
    ax.set_xlabel("Theoretical Quantiles")
    ax.set_ylabel("Standardized Residuals")
    return ax

def scale_location_plot(self, ax=None):
    """
    Sqrt(Standarized Residual) vs Fitted values plot

    Used to check homoscedasticity of the residuals.
    Horizontal line will suggest so.
    """
    if ax is None:
        fig, ax = plt.subplots()

    residual_norm_abs_sqrt = np.sqrt(np.abs(self.residual_norm))

    ax.scatter(self.y_predict, residual_norm_abs_sqrt, alpha=0.5)
    sns.regplot(
        x=self.y_predict,
        y=residual_norm_abs_sqrt,
        scatter=False,

```

```

        ci=False,
        lowess=True,
        line_kws={"color": "red", "lw": 1, "alpha": 0.8},
        ax=ax,
    )

    # annotations
    abs_sq_norm_resid = np.flip(np.argsort(residual_norm_abs_sqrt), 0)
    abs_sq_norm_resid_top_3 = abs_sq_norm_resid[:3]
    for i in abs_sq_norm_resid_top_3:
        ax.annotate(
            i, xy=(self.y_predict[i], residual_norm_abs_sqrt[i]), color="C3"
        )

    ax.set_title("Scale-Location", fontweight="bold")
    ax.set_xlabel("Fitted values")
    ax.set_ylabel(r"$\sqrt{|\mathrm{Standardized\ Residuals}|}$")
    return ax

def leverage_plot(
    self, ax=None, high_leverage_threshold=False, cooks_threshold="baseR"
):
    """
    Residual vs Leverage plot

    Points falling outside Cook's distance curves are considered
    ↪ observation that can sway the fit
    aka are influential.
    Good to have none outside the curves.
    """
    if ax is None:
        fig, ax = plt.subplots()

    ax.scatter(self.leverage, self.residual_norm, alpha=0.5)

    sns.regplot(
        x=self.leverage,
        y=self.residual_norm,
        scatter=False,
        ci=False,
        lowess=True,
        line_kws={"color": "red", "lw": 1, "alpha": 0.8},
        ax=ax,
    )

    # annotations
    leverage_top_3 = np.flip(np.argsort(self.cooks_distance), 0)[:3]

```

```

    for i in leverage_top_3:
        ax.annotate(i, xy=(self.leverage[i], self.residual_norm[i]),
↪color="C3")

    factors = []
    if cooks_threshold == "baseR" or cooks_threshold is None:
        factors = [1, 0.5]
    elif cooks_threshold == "convention":
        factors = [4 / self.nresids]
    elif cooks_threshold == "dof":
        factors = [4 / (self.nresids - self.nparams)]
    else:
        raise ValueError(
            "threshold_method must be one of the following: 'convention',
↪'dof', or 'baseR' (default)"
        )
    for i, factor in enumerate(factors):
        label = "Cook's distance" if i == 0 else None
        xtemp, ytemp = self.__cooks_dist_line(factor)
        ax.plot(xtemp, ytemp, label=label, lw=1.25, ls="--", color="red")
        ax.plot(xtemp, np.negative(ytemp), lw=1.25, ls="--", color="red")

    if high_leverage_threshold:
        high_leverage = 2 * self.nparams / self.nresids
        if max(self.leverage) > high_leverage:
            ax.axvline(
                high_leverage, label="High leverage", ls="-. ",
↪color="purple", lw=1
            )

    ax.axhline(0, ls="dotted", color="black", lw=1.25)
    ax.set_xlim(0, max(self.leverage) + 0.01)
    ax.set_ylim(min(self.residual_norm) - 0.1, max(self.residual_norm) + 0.
↪1)

    ax.set_title("Residuals vs Leverage", fontweight="bold")
    ax.set_xlabel("Leverage")
    ax.set_ylabel("Standardized Residuals")
    plt.legend(loc="best")
    return ax

def vif_table(self):
    """
    VIF table

    VIF, the variance inflation factor, is a measure of multicollinearity.
    VIF > 5 for a variable indicates that it is highly collinear with the
    other input variables.

```

```

        """
        vif_df = pd.DataFrame()
        vif_df["Features"] = self.xvar_names
        vif_df["VIF Factor"] = [
            variance_inflation_factor(self.xvar, i) for i in range(self.xvar.
↪shape[1])
        ]

        return vif_df.sort_values("VIF Factor").round(2)

    def __cooks_dist_line(self, factor):
        """
        Helper function for plotting Cook's distance curves
        """
        p = self.nparams
        formula = lambda x: np.sqrt((factor * p * (1 - x)) / x)
        x = np.linspace(0.001, max(self.leverage), 50)
        y = formula(x)
        return x, y

    def __qq_top_resid(self, quantiles, top_residual_indices):
        """
        Helper generator function yielding the index and coordinates
        """
        offset = 0
        quant_index = 0
        previous_is_negative = None
        for resid_index in top_residual_indices:
            y = self.residual_norm[resid_index]
            is_negative = y < 0
            if previous_is_negative == None or previous_is_negative == ↪
↪is_negative:
                offset += 1
            else:
                quant_index -= offset
            x = (
                quantiles[quant_index]
                if is_negative
                else np.flip(quantiles, 0)[quant_index]
            )
            quant_index += 1
            previous_is_negative = is_negative
            yield resid_index, x, y

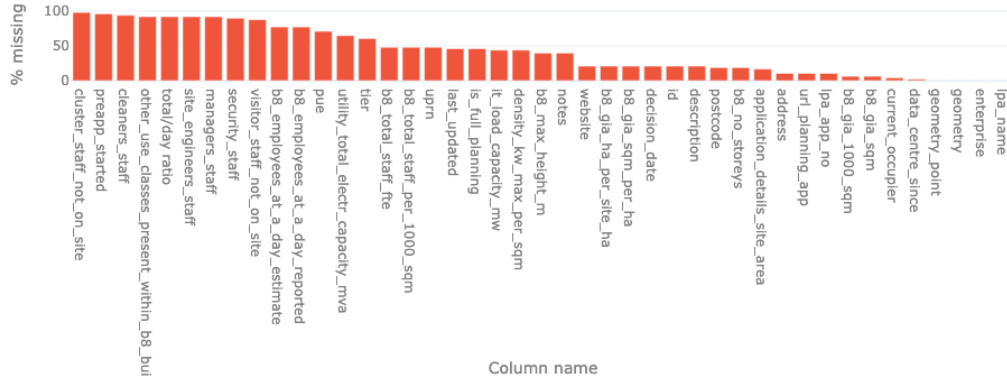
notes_colour = "black"
labels_colour = "grey"

```

```
[8]: #
#
# Analysing the column NA values
#
missing_pct = (data_ldn_exstg.isnull().mean() * 100).
    sort_values(ascending=False)
fig = px.bar(missing_pct, labels={"value": "% missing", "index": "Column name"},
    color_discrete_sequence=[px.colors.qualitative.Plotly[1]])
fig.update_layout(width=1000, height=400, template="plotly_white",
    showlegend=False,
    title=dict(
        text=(
            f"<span style='font-size:24px; font-weight:bold; color:
    {notes_colour}'; display:block; margin-bottom:2px;'>"
            "Missing values by column (%)"
            "</span><br>"
        ),
        x=0.03, xanchor="left", y=0.92, yanchor="top",
    ),
    margin=dict(t=90), legend=dict(font=dict(color=labels_colour)),
)

fig.
    update_xaxes(title_font=dict(color=labels_colour), tickfont=dict(color=labels_colour))
fig.
    update_yaxes(title_font=dict(color=labels_colour), tickfont=dict(color=labels_colour))
fig.show()
#
#
# The dataset has a lot of NA values, and it will present an obstacle for
    clustering. A more "full" columns are therefore to be chosen
#
```

Missing values by column (%)



```
[8]: value_cols = ['b8_total_staff_fte', 'b8_gia_1000_sqm',
    ↪ 'application_details_site_area', 'b8_total_staff_per_1000_sqm',
    ↪ 'b8_gia_sqm_per_ha', 'b8_gia_ha_per_site_ha',
    ↪ "utility_total_electr_capacity_mva",
    ↪ "it_load_capacity_mw", "density_kw_max_per_sqm"]
# -----
# Correlation matrix
# -----
corr_cols = value_cols
df = data_ldn_exstg[corr_cols].copy()
corr = df.corr(numeric_only=True)

fig = go.Figure(go.Heatmap(z=corr.values,x=corr.columns,y=corr.
    ↪ index, colorscale="RdBu", zmin=-1, zmax=1,
    text=corr.round(2).values, texttemplate="%{text}", hovertemplate="X:
    ↪ %{x}<br>Y: %{y}<br>Correlation: %{z:.2f}<extra></extra>",
    colorbar=dict( title="Correlation", len=0.65, thickness=12, x=1.02, ),
))

fig.update_layout(
    width=1000, height=600, template="plotly_white",
    title=dict(
        text=(
            f"<br><span style='font-size:24px; font-weight:bold; color:
            ↪ {notes_colour}; display:block; margin-bottom:2px;'>"
            "Correlation matrix (numeric variables)"
            "</span><br>"
            f"<span style='font-size:13px; font-weight:300; color:
            ↪ {notes_colour}; display:block; margin-bottom:0px;'>"
            "Shows linear correlation between staffing, size and derived
            ↪ variables."
```



```

        "</span><br>"
    ),x=0.03,xanchor="left",y=0.92,yanchor="top",
    ),margin=dict(t=160),
)
fig.update_xaxes(tickangle=25, tickfont=dict(color=labels_colour),
    ↪title_font=dict(color=labels_colour))
fig.update_yaxes(tickfont=dict(color=labels_colour),
    ↪title_font=dict(color=labels_colour))
fig.show()

```

1.3 Section 1: Data Centre Segmentation

Recent research by the House Of Commons outlines four DC types (see main report). The dataset contains several variables that could inform clustering of DC, but few are uncorrelated with GIA. Since the interaction terms is used later in the document, the clustering and independent variable should not come from the same source as GIA, the planned independent variable. We have tried a variety of combinations of the clustering based on technical parameters, but these clusters performed poorly in regression with interaction terms.

For this reason, spatial clustering variable was introduced using DBSCAN. To identify distinct groups, a combination of elbow method and visual analysis was used.

```

[9]: # -----
# Elbow method - selecting eps=5000
# -----
# use meters
g = data_ldn_exstg.to_crs(27700).copy()
X = np.column_stack([g.geometry.x, g.geometry.y])

k = 4 # usually = min_samples
nbrs = NearestNeighbors(n_neighbors=k).fit(X)
distances, _ = nbrs.kneighbors(X)

# distance to kth nearest neighbour
k_dist = np.sort(distances[:, -1])

fig = go.Figure(go.Scatter(y=k_dist, mode="lines", name=f"{k}-NN distance"))
fig.update_layout(
    template="plotly_white",
    width=1000,
    height=500,
    title=f"DBSCAN eps selection: {k}-NN distance plot",
    xaxis_title="Points (sorted)",
    yaxis_title=f"Distance to {k}th nearest neighbour (meters)",
)
fig.show()

```

```

[10]: # -----
# Clustering
# -----
#meters
g = data_ldn_exstg.to_crs(27700).copy()
mask = g.geometry.notna()
X = np.column_stack([g.loc[mask].geometry.x, g.loc[mask].geometry.y])

# fit DBSCAN
db = DBSCAN(eps=5000, min_samples=3) # <- tune eps
labels = db.fit_predict(X)

# write labels back
data_ldn_exstg["cluster"] = np.nan
data_ldn_exstg.loc[g.loc[mask].index, "cluster"] = labels
data_ldn_exstg["cluster"] = data_ldn_exstg["cluster"].astype("Int64")

# Rename the clusters for clarity
data_ldn_exstg["cluster_text"] = np.select(
    [
        data_ldn_exstg["cluster"] == 0,
        data_ldn_exstg["cluster"] == 1,
        data_ldn_exstg["cluster"] == 2,
        data_ldn_exstg["cluster"] == 3,
        data_ldn_exstg["cluster"] == -1,
    ],
    ["Central", "Hillingdon", "Park Royal", "Enfield", "Fringe Areas"],
    default=None
)

# -----
# Plotting
# -----

palette = px.colors.qualitative.G10
clusters_sorted = sorted(np.unique(labels))
cluster_colour_map = {cl: palette[i % len(palette)] for i, cl in enumerate(clusters_sorted)}

g4326 = data_ldn_exstg.to_crs(4326).copy()
g4326["lon"], g4326["lat"] = g4326.geometry.x, g4326.geometry.y
g4326["cluster"] = g4326["cluster"].astype(str)

gla = gla_boundary.to_crs(4326).copy()
gla_xy = gla.geometry.iloc[0].boundary.xy

fig = px.scatter_map(

```

```

    g4326, lat="lat", lon="lon", color="cluster_text",
    ↪color_discrete_sequence=palette, labels={"cluster": "Indicative cluster",
    ↪name"},
    map_style="carto-positron", zoom=8,
    ↪hover_data=["id", "lpa_name", "current_occupier"]
)

fig.add_trace(go.Scattermap(
    lon=list(gla_xy[0]), lat=list(gla_xy[1]),
    mode="lines", line=dict(color="black", width=1),
    name="GLA boundary", hoverinfo="skip"
))

fig.update_layout(
    map=dict(
        domain=dict(x=[0, 1], y=[0, 1]),
        center=dict(lat=51.50758519898516, lon=-0.1181236104918927),
        zoom=9
    ),
    margin=dict(l=0, r=0, t=90, b=0)
)

fig.update_layout(
    width=900, height=700, template="plotly_white",
    title=dict(
        text=(
            "<span style='font-size:24px; font-weight:bold; color:black;
            ↪display:block;'>"
            "Existing data centre clusters"
            "</span><br>"
            "<span style='font-size:13px; font-weight:300; color:black; display:
            ↪block;'>"
            "Based on DBSCAN clustering, using bespoke sample of London data,
            ↪centres."
            "</span>"
        ),
        x=0.00, xanchor="left", y=0.95, yanchor="top"
    ),
    margin=dict(t=90), legend=dict(font=dict(color="grey")),
    map=dict(center=dict(lat=51.50758519898516, lon=-0.09281236104918927))
)
fig.update_traces(marker=dict(size=10))
fig.show()

```

```

[12]: #
    ↪#
    # # REDUNDANT CODE - LEFT in to demonstrate the continuity of thinking

```

```

#_
↳#-----

# clustering_by = ['density_kw_max_per_sqm'] #'density_kw_max_per_sqm',_
↳it_load_capacity_mw',
# data_sizes_no_na = data_ldn_exstg[clustering_by]
# data_sizes_no_na = data_sizes_no_na.dropna(subset=clustering_by)
# #_
↳-----

# # the imputation was removed to minimise collinearity issues, since majority_
↳of variables in the dataset are related to each other
# #_
↳-----

# # Use K-nearest neighbours imputation (k = 5)
# # I am filling MW and KW columns, but not the GIA since it is most filled and_
↳most reliable. Also, for most null GIA, the other columns are null as well
# # imputer = KNNImputer(n_neighbors=5)
# # data_sizes_imputed_array = imputer.fit_transform(data_sizes_no_na)
# # data_sizes_no_na = pd.DataFrame(data_sizes_imputed_array, columns=[
# #     "b8_gia_sqm",
# #     "it_load_capacity_mw"], index=data_sizes_no_na.index)

# scaler = StandardScaler()
# scaled_array = scaler.fit_transform(data_sizes_no_na)
# data_sizes_no_na = pd.
↳DataFrame(scaled_array, columns=clustering_by, index=data_sizes_no_na.index)
# #_
↳-----

# # elbow plot
# #_
↳-----

# X = data_sizes_no_na[clustering_by]
# inertias = []
# k_values = list(range(2, 11))

# for k in k_values:
#     kmeans = KMeans(n_clusters=k, random_state=42, n_init="auto")
#     kmeans.fit(X)
#     inertias.append(kmeans.inertia_)

# fig = go.Figure()
# fig.add_trace(go.
↳Scatter(x=k_values, y=inertias, mode="lines+markers", name="Inertia",_
↳line=dict(color=px.colors.qualitative.Plotly[1]), marker=dict(color=px.
↳colors.qualitative.Plotly[1])))
# fig.update_layout(

```

```

#     width=1000,height=500, template="plotly_white",
#     title=dict(
#         text=(
#             f"<span style='font-size:24px; font-weight:bold; color:
↳{notes_colour}; display:block; margin-bottom:2px;'>"
#             "Elbow plot: Identifying optimal number of clusters"
#             "</span><br>"
#         ),
#         x=0.03,xanchor="left", y=0.92,yanchor="top",
#     ),
#     margin=dict(t=90), legend=dict(font=dict(color=labels_colour)),
#     xaxis_title="Number of clusters",yaxis_title="Inertia (within-cluster sum
↳of squares)",
# )

# fig.
↳update_xaxes(title_font=dict(color=labels_colour),tickfont=dict(color=labels_colour))
# fig.
↳update_yaxes(title_font=dict(color=labels_colour),tickfont=dict(color=labels_colour))
# fig.show()

# #
↳
# # clustering
# #
↳
# k_means_optimum = KMeans(n_clusters = 4, init = 'k-means++', random_state=42,
↳n_init="auto")
# y = k_means_optimum.fit_predict(X)
# # print(y)

# # write clusters into the initial dataframe
# data_ldn_exstg["cluster"] = np.nan
# data_ldn_exstg.loc[data_sizes_no_na.index, "cluster"] = y
# sil_score = silhouette_score(X,y)
# print(sil_score)

# #
↳
# # Diagnostics: cardinality, magnitude, silhouette plots
# #
↳

# # https://medium.com/@sk.shravan00/k-means-for-3-variables-260d20849730
# # https://towardsdatascience.com/
↳best-practices-for-visualizing-your-cluster-results-20a3baac7426/

```

```

# df_plot = data_ldn_exstg.copy()

# xcol = "density_kw_max_per_sqm"
# ycol = "it_load_capacity_mw"
# clusters_sorted = sorted(df_plot["cluster"].dropna().unique())
# palette = px.colors.qualitative.G10
# cluster_colour_map = {cl: palette[i % len(palette)] for i, cl in
    ↪ enumerate(clusters_sorted)}

# cols_to_show = ["id", "current_occupier", "lpa_name", "data_centre_since",
    ↪ "b8_gia_sqm", "b8_total_staff_fte", xcol, ycol]
# hovertemplate = "<br>".join([f"{c}: {{{customdata[{i}]}}}" for i, c in
    ↪ enumerate(cols_to_show)])
# sil_vals = silhouette_samples(X, y)
# sil_score = silhouette_score(X, y)

# fig = make_subplots(rows=1, cols=2, column_widths=[0.65, 0.35],
    ↪ horizontal_spacing=0.08, subplot_titles=["K-means clusters", "Silhouette
    ↪ plot"],)
# #
    ↪ -----
# # LEFT: Scatterplot clusters
# for i, cl in enumerate(clusters_sorted):
#     d = df_plot[df_plot["cluster"] == cl]
#     fig.add_trace(go.Scatter(
#         x=d[xcol], y=d[ycol], mode="markers", marker=dict(size=6,
    ↪ color=cluster_colour_map[cl], opacity=0.9),
#         name=f"Cluster {int(cl)}", customdata=d[cols_to_show].
    ↪ to_numpy(), hovertemplate=hovertemplate,
#         ), row=1, col=1
#     )

# # # Cluster centres
# # centres_scaled = k_means_optimum.cluster_centers_
# # centres_unscaled = scaler.inverse_transform(centres_scaled)

# # fig.add_trace(go.Scatter(
# #     x=centres_unscaled[:, 0], y=centres_unscaled[:, 1],
# #     mode="markers+text", marker=dict(size=10, symbol="diamond",
    ↪ color="black"),
# #     text=[f"GIA:{x:,.0f}, MW:{y:,.1f}" for x, y in zip(centres_unscaled[:,
    ↪ 0], centres_unscaled[:, 1])],
# #     textposition="top center", textfont=dict(color="black", size=10),
# #     name="Cluster centres"
# # ))
# # # Overlay: enterprise=True (small black dot on top)

```

```

# # enterprise_mask = (df_plot["enterprise"] == True)
# # # & df_plot["cluster"].notna()

# # fig.add_trace(go.Scatter(
# #     x=df_plot.loc[enterprise_mask, xcol], y=df_plot.loc[enterprise_mask,
# #     ↪ ycol],
# #     mode="markers", marker=dict(size=14,
# #     ↪ symbol="circle-open", line=dict(color="black", width=2)),
# #     name="Enterprise", showlegend=True, hoverinfo="skip"
# #     ), row=1, col=1
# # )
# # ↪
# # -----
# # RIGHT: Silhouette plot
# y_lower = 0

# for i, cl in enumerate(sorted(np.unique(y))):
#     cl_vals = sil_vals[y == cl]
#     cl_vals.sort()
#     y_upper = y_lower + len(cl_vals)
#     y_range = np.arange(y_lower, y_upper)

#     # filled silhouette area
#     fig.add_trace(go.Scatter(
#     ↪ x=cl_vals, y=y_range, mode="lines", line=dict(width=1,
#     ↪ color=cluster_colour_map[cl]), fill="tozerox",
#     ↪ fillcolor=make_transparent(cluster_colour_map.get(cl), alpha=0.25),
#     name=f"Cluster {cl}", showlegend=False, # keep legend only for
#     ↪ left plot
#     ↪ hovertemplate=(
#     ↪     f"Cluster {cl}<br>"
#     ↪     "Silhouette: %{x:.3f}<extra></extra>"
#     ↪ ),
#     ↪ ), row=1, col=2
#     ↪ )
#     # label at centre of cluster
#     fig.add_trace(go.Scatter(x=[-0.05], y=[y_lower + 0.5 *
#     ↪ len(cl_vals)], mode="text", text=[str(cl)], showlegend=False, hoverinfo="skip",), row=1,
#     ↪ col=2)
#     y_lower = y_upper + 10 # spacing between clusters

# # Mean silhouette score line
# fig.
#     ↪ add_vline(x=sil_score, line_width=2, line_dash="dash", line_color="red", row=1,
#     ↪ col=2)

```

```

# #
# # Layout
# fig.update_layout(
#     width=1300, height=650, template="plotly_white",
#     title=dict(
#         text=(
#             f"<span style='font-size:24px; font-weight:bold; color:
↳ {notes_colour}; display:block;'>"
#             "Cluster diagnostics: K-means clusters and silhouette"
#             "</span><br>"
#             f"<span style='font-size:13px; font-weight:300; color:
↳ {notes_colour}; display:block;'>"
#             f"Mean silhouette score = {sil_score:.3f}"
#             "</span>"
#         ),
#         x=0.03, xanchor="left", y=0.95, yanchor="top"
#     ), margin=dict(t=90), legend=dict(font=dict(color=labels_colour)),
# )

# fig.update_annotations(font=dict(size=14, color=labels_colour))
# fig.update_xaxes( title_text=xcol, row=1, col=1,
↳ title_font=dict(color=labels_colour), tickfont=dict(color=labels_colour),)
# fig.update_yaxes( title_text=ycol, row=1,
↳ col=1, title_font=dict(color=labels_colour), tickfont=dict(color=labels_colour),)
# fig.update_xaxes( title_text="Silhouette coefficient", row=1, col=2, range=[-0.
↳ 2, 1.0], title_font=dict(color=labels_colour),
↳ tickfont=dict(color=labels_colour),)
# fig.update_yaxes( title_text="Observations (stacked by cluster)", row=1,
↳ col=2, title_font=dict(color=labels_colour),
↳ tickfont=dict(color=labels_colour), showgrid=False)
# fig.show()

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[12], line 19
      7 data_sizes_no_na = data_sizes_no_na.dropna(subset=clustering_by)
      8 #
↳
      9 # the imputation was removed to minimise collinearity issues, since
↳ majority of variables in the dataset are related to each other
     10 #
↳
     (...)    16 #         "b8_gia_sqm",
     17 #         "it_load_capacity_mw"], index=data_sizes_no_na.index)
--> 19 scaler = StandardScaler()

```



```

20 scaled_array = scaler.fit_transform(data_sizes_no_na)
21 data_sizes_no_na = pd.
↳ DataFrame(scaled_array, columns=clustering_by, index=data_sizes_no_na.index)

NameError: name 'StandardScaler' is not defined

```

1.3.1 Cluster Diagnostics

```

[12]: #
↳
# Cardinality plot only (points per cluster)
#
↳
# labels = np.array(y)
cardinality = pd.Series(labels).value_counts().sort_index()

fig = go.Figure(go.Bar(
    x=[f"Cluster {cl}" for cl in cardinality.index],
    y=cardinality.values,
    marker=dict(color=[make_transparent(cluster_colour_map[cl], alpha=0.25) for
↳ cl in cardinality.index]),
    showlegend=False,
    hovertemplate="Cluster: %{x}<br>N: %{y}<extra></extra>",
))

fig.update_layout(
    width=900, height=450, template="plotly_white",
    title=dict(
        text=(
            f"<span style='font-size:24px; font-weight:bold; color:
↳ {notes_colour}; display:block;'>"
            "Cardinality bar plot"
            "</span><br>"
            f"<span style='font-size:13px; font-weight:300; color:
↳ {notes_colour}; display:block;'>"
            "Cardinality = number of points in each cluster."
            "</span>"
        ),
        x=0.03, xanchor="left", y=0.95, yanchor="top"
    ),
    margin=dict(t=90),
)

fig.update_xaxes(title_text="Clusters", title_font=dict(color=labels_colour),
↳ tickfont=dict(color=labels_colour))

```

```
fig.update_yaxes(title_text="Count of observations (N)",
                 title_font=dict(color=labels_colour), tickfont=dict(color=labels_colour))

fig.show()
```

We observe clusters based on location density:

- **Cluster 0: Central London.** Most observations, sufficient for regression.
- **Clusters 1,2,3,-1.** Insufficient observations, high chance of cluster loss when overlapping with x/y variables.

To preserve the most number of datapoints, we joined the non-central clusters together.

```
[13]: #
# -----
# reassign clusters
#
# -----
data_ldn_exstg["cluster_text"] = np.select(
    [
        data_ldn_exstg["cluster"] == 0,
        data_ldn_exstg["cluster"] == 1,
        data_ldn_exstg["cluster"] == 2,
        data_ldn_exstg["cluster"] == 3,
        data_ldn_exstg["cluster"] == -1,
    ],
    ["Central", "Fringe", "Fringe", "Fringe", "Fringe"],
    default=None
)
```

1.4 Exploring the Link Between Data Centre Size and Direct Employment Generated

In this section, we perform regression analysis. Below are some aspects that determine the choice of variables and methods:

- The theme is not well-researched, and there are no assumptions on what can drive employment levels in DC. Basic assumption in this study is that the larger the facility is (**b8_gia_sqm**), the more staff it requires;
- The dataset has gaps, and to study the size-employment link, we need to drop the null values in both columns, as well as the outliers.
- Given the uneven N of observations in clusters, some of them will be underrepresented, but this was addressed in the clustering section.

1.4.1 Baseline modelling

```
[14]: #  
      ↪  
      # Checking the distributions  
      #  
      ↪  
      value_cols = ["data_centre_since", "b8_gia_sqm", "b8_total_staff_fte",  
                    ↪ "b8_employees_at_a_day_estimate", "utility_total_electr_capacity_mva",  
                    ↪ "it_load_capacity_mw", "density_kw_max_per_sqm"]  
      cols_to_show = ['id', 'lpa_name', 'current_occupier'] + value_cols  
  
      df0 = data_ldn_exstg.copy()  
  
      n = len(value_cols)  
      ncols = 3  
      nrows = math.ceil(n / ncols)  
      base_color = px.colors.qualitative.Plotly[1]  
      fig = make_subplots(rows=nrows, cols=ncols, horizontal_spacing=0.  
                    ↪ 05, vertical_spacing=0.12)  
      # -----  
      # Violin plots  
      # -----  
      for idx, value_col in enumerate(value_cols):  
          r = idx // ncols + 1  
          c = idx % ncols + 1  
          df = df0.dropna(subset=[value_col]).copy()  
          if df.empty:  
              continue  
  
          subset_customdata = df[cols_to_show].to_numpy()  
          hovertemplate = "<br>".join([f"{col}: %{{customdata[{i}]}}" for i, col in  
                    ↪ enumerate(cols_to_show)])  
          median_value = df[value_col].median()  
  
          # # IQR bounds. Using the IQR, the outlier data points are the ones falling  
                    ↪ below Q1-1.5 IQR or above Q3 + 1.5 IQR: https://careerfoundry.com/en/blog/  
                    ↪ data-analytics/how-to-find-outliers/  
          q1x, q3x = df[value_col].quantile([0.25, 0.75])  
          iqr_x = q3x - q1x  
          low_x, high_x = q1x - 1.5 * iqr_x, q3x + 1.5 * iqr_x  
          iqr_bounds = [low_x, high_x]  
  
          # Auto bandwidth based on numeric range  
          vmin, vmax = df[value_col].min(), df[value_col].max()  
          rng = vmax - vmin  
          if pd.isna(rng) or rng == 0:
```

```

        bw_bottom, bw_top = 1, 3
    else:
        bw_bottom = max(rng * 0.03, rng * 0.005)
        bw_top = max(rng * 0.10, rng * 0.02)

    fig.add_trace(go.Violin(
        x=df[value_col],orientation='h', side='positive', width=2,
        ↪bandwidth=bw_top,line=dict(width=0),
        points="all",jitter=0.3, pointpos=-0.4, marker=dict(color='grey',
        ↪size=3, opacity=1),
        box_visible=True, box_fillcolor='black', box_line=dict(color='black',
        ↪width=3), box_width=0.2,
        fillcolor=base_color, showlegend=False, customdata=subset_customdata,
        ↪hovertemplate=hovertemplate,
        ), row=r, col=c)
# -----
# Helper elements
# -----
    # IQR bounds
    for i, b in enumerate(iqr_bounds):
        fig.add_vline(x=b, line_width=1, line_dash="dot", line_color="black",
        ↪row=r, col=c, showlegend=False)
        # invisible point just for hover
        fig.add_trace(go.Scatter(x=[b], y=[0],mode="markers",
        ↪marker=dict(opacity=0),hovertemplate=f"IQR bound: {b:,.2f}<extra></
        ↪extra>",showlegend=False,), row=r, col=c)

    # Median arrow
    fig.add_trace(go.Scatter(x=[median_value], mode='markers',
    ↪marker=dict(symbol='arrow-down', color='black', size=12), showlegend=False,
    ↪hovertemplate="<br>".join([f"Median: {median_value}"])), row=r, col=c)

    # Style subplot axes
    fig.update_yaxes(showticklabels=False, title="Density (KDE)", ticks="",
    ↪showgrid=False, zeroline=False, row=r, col=c)
    fig.update_xaxes(title_text=value_col, row=r, col=c)
    fig.update_xaxes(tickangle=25, tickfont=dict(color=labels_colour),
    ↪title_font=dict(color=labels_colour))
    fig.update_yaxes(tickfont=dict(color=labels_colour),
    ↪title_font=dict(color=labels_colour))

fig.update_layout(violinmode='overlay', width=1400, height=250 * nrows,
    ↪template='plotly_white',
    title=dict(
        text=(

```

```

        f"<br><span style='font-size:24px; font-weight:bold; color:
↳{notes_colour}; display:block; margin-bottom:2px;'>"
        "Candidate variables' distribution"
        "</span><br>"
        f"<span style='font-size:13px; font-weight:300; color:
↳{notes_colour}; display:block; margin-bottom:0px;'>"
        "All plots show long tails with notable outliers (points beyond
↳dashed IQR bounds)."
```

```

        "</span><br>"
    ),
    x=0.03, xanchor="left", y=0.92, yanchor="top",
),
margin=dict(t=150),
)
fig.show()
```

Relevant distributions have long right tails with severe outliers. Due to the low number of observations, we are applying a sequential approach to modelling, and aiming to have 1 explanatory variable per model. Outliers are identified and dropped individually for the model to maximise the number of datapoints. For the baseline OLS, the outliers are giving too much variance, which results in statistically insignificant slope and intercept. We will aim to address them as part of the Interaction Term model.

```

[15]: #
↳
# testing alternative independent variables
#
↳
y_col = "b8_total_staff_fte"
x_cols = ["b8_gia_1000_sqm", "application_details_site_area",
↳"b8_total_staff_per_1000_sqm", "b8_gia_sqm_per_ha", "b8_gia_ha_per_site_ha"]
results = []
for x_col in x_cols:
    value_cols = [y_col, x_col]
    d = data_ldn_exstg.dropna(subset=value_cols).copy()
    if d.empty or len(d) < 8:
        continue
    #
↳
    # outlier removal using IQR
    for value_col in value_cols:
        q1, q3 = d[value_col].quantile([0.25, 0.75])
        iqr = q3 - q1
        low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
        d = d[d[value_col].between(low, high)]
    if d.empty or len(d) < 8:
        continue
```

```

#
# OLS
y = d[y_col]
X = sm.add_constant(d[x_col])
model = sm.OLS(y, X).fit()
#
# store model results
results.append({
    "x_var": x_col,
    "n": int(model.nobs),
    "beta": model.params[x_col],
    "p_value": model.pvalues[x_col],
    "R2": model.rsquared,
    "Adj_R2": model.rsquared_adj,
    "AIC": model.aic,
    "BIC": model.bic,
    "intercept": model.params["const"],
})
compare_models_df = (pd.DataFrame(results).sort_values("p_value").
    ↪reset_index(drop=True))
compare_models_df

```

```

[15]:

```

	x_var	n	beta	p_value	R2	Adj_R2	\
0	b8_gia_1000_sqm	20	2.238384	0.000987	0.461482	0.431565	
1	application_details_site_area	20	9.791580	0.169367	0.102233	0.052357	
2	b8_total_staff_per_1000_sqm	21	-0.843189	0.545351	0.019566	-0.032036	
3	b8_gia_sqm_per_ha	18	0.000260	0.725768	0.007902	-0.054104	
4	b8_gia_ha_per_site_ha	18	2.598798	0.725768	0.007902	-0.054104	

	AIC	BIC	intercept
0	188.101435	190.092900	20.485779
1	199.122566	201.114030	33.541935
2	217.635009	219.724054	56.210659
3	188.807819	190.588562	47.405063
4	188.807819	190.588562	47.405063

In univariate OLS screening, **only b8_gia_1000_sqm showed a statistically significant relationship with total staff numbers**. Other candidate predictors were not significant, and had substantially lower explanatory power, so they were excluded from further diagnostics and modelling.

```

[16]: #
# Baseline Model: b8_total_staff_fte ~ b8_gia_1000_sqm
value_cols_m1 = ['b8_total_staff_fte', 'b8_gia_1000_sqm']

```

```

data_ldn_exstg_regression_m1 = data_ldn_exstg.dropna(subset=value_cols_m1).
↳copy()

#_
↳#_
# # Dropping outliers using IQR. https://careerfoundry.com/en/blog/
↳data-analytics/how-to-find-outliers/
#_
↳#_
for value_col in value_cols_m1:
    #no outliers
    q1x, q3x = data_ldn_exstg_regression_m1[value_col].quantile([0.25, 0.75])
    iqr_x = q3x - q1x
    low_x, high_x = q1x - 1.5 * iqr_x, q3x + 1.5 * iqr_x
    data_ldn_exstg_regression_m1 = _
↳data_ldn_exstg_regression_m1[data_ldn_exstg_regression_m1[value_col].
↳between(low_x, high_x)]

    # #log-transformation for testing with both
    # data_ldn_exstg_regression_m1[f"log_{value_col}"] = np.
↳log(data_ldn_exstg_regression_m1[value_col]+ 1e-6)

y = data_ldn_exstg_regression_m1["b8_total_staff_fte"]
X = sm.add_constant(data_ldn_exstg_regression_m1["b8_gia_1000_sqm"])

m1 = sm.OLS(y, X).fit()
m1_robust_se = sm.OLS(y, X).fit(cov_type="HC3") #a model variation with robust_
↳standard errors - solution in the presence of heteroscedasticity
m1_robust_HuberT = sm.RLM(y, X, M=sm.robust.norms.HuberT()).fit() #a model_
↳variation that downweights outliers
print(m1.summary())
print(m1_robust_se.summary())
print(m1_robust_HuberT.summary())

```

OLS Regression Results

```

=====
Dep. Variable:    b8_total_staff_fte    R-squared:                0.461
Model:            OLS                  Adj. R-squared:           0.432
Method:           Least Squares        F-statistic:             15.43
Date:             Tue, 25 Aug 2026     Prob (F-statistic):      0.000987
Time:            22:57:20              Log-Likelihood:          -92.051
No. Observations: 20                  AIC:                     188.1
Df Residuals:     18                  BIC:                     190.1
Df Model:         1
Covariance Type:  nonrobust
=====

```

```

===
              coef      std err          t      P>|t|      [0.025
0.975]
-----
---
const          20.4858      8.473      2.418      0.026      2.684
38.287
b8_gia_1000_sqm  2.2384      0.570      3.927      0.001      1.041
3.436
=====
Omnibus:                0.183    Durbin-Watson:                1.665
Prob(Omnibus):          0.912    Jarque-Bera (JB):          0.141
Skew:                   0.154    Prob(JB):                  0.932
Kurtosis:               2.728    Cond. No.                  22.2
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Regression Results

```

=====
Dep. Variable:    b8_total_staff_fte    R-squared:                0.461
Model:            OLS                  Adj. R-squared:            0.432
Method:           Least Squares         F-statistic:              24.25
Date:             Tue, 25 Aug 2026      Prob (F-statistic):       0.000109
Time:             22:57:20              Log-Likelihood:           -92.051
No. Observations: 20                  AIC:                      188.1
Df Residuals:     18                  BIC:                      190.1
Df Model:         1
Covariance Type:  HC3
=====

```

```

===
              coef      std err          z      P>|z|      [0.025
0.975]
-----
---
const          20.4858      7.675      2.669      0.008      5.444
35.528
b8_gia_1000_sqm  2.2384      0.455      4.924      0.000      1.347
3.129
=====
Omnibus:                0.183    Durbin-Watson:                1.665
Prob(Omnibus):          0.912    Jarque-Bera (JB):          0.141
Skew:                   0.154    Prob(JB):                  0.932
Kurtosis:               2.728    Cond. No.                  22.2
=====

```

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

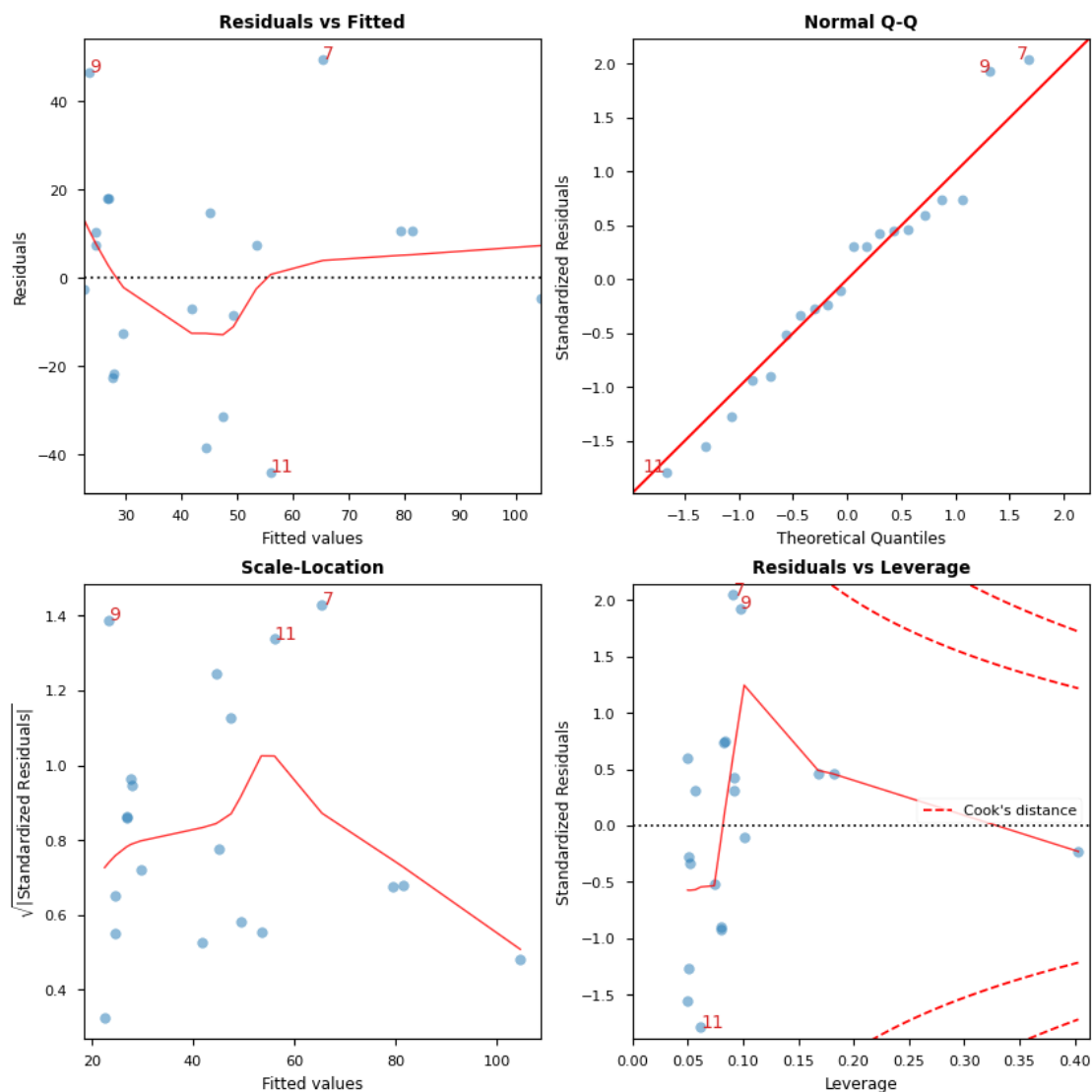
Robust linear Model Regression Results

```
=====
Dep. Variable:      b8_total_staff_fte    No. Observations:      20
Model:              RLM                  Df Residuals:          18
Method:             IRLS                 Df Model:              1
Norm:               HuberT
Scale Est.:         mad
Cov Type:           H1
Date:               Tue, 25 Aug 2026
Time:               22:57:20
No. Iterations:     9
=====
===
               coef      std err          z      P>|z|      [0.025
0.975]
-----
---
const          19.4759      8.606      2.263      0.024      2.608
36.344
b8_gia_1000_sqm  2.2747      0.579      3.930      0.000      1.140
3.409
=====
===
```

If the model instance has been used for another fit with different fit parameters, then the fit options might not be the correct ones anymore .

```
[17]: cls1 = LinearRegDiagnostic(m1)
vif1, fig1, ax = cls1()
print(vif1)

#1. Horizontal red line is an indicator that the residual has a linear pattern
#2. Points spread along the diagonal line will suggest normality of residuals
#3. Used to check homoscedasticity of the residuals. Horizontal line will
    ↳ suggest so.
#4. Points falling outside Cook's distance curves are considered observation
    ↳ that can sway the fit aka are influential. Good to have none outside the
    ↳ curves.
```



	Features	VIF	Factor
1	b8_gia_1000_sqm	1.00	
0	const	2.22	

- 1) Residuals vs Fitted plot: potential nonlinearity, the model is systematically wrong in the mid fitted range.
- 2) Q-Q plot: positive residuals on the higher end.
- 3) Scale-Location: non-constant variance (heteroskedasticity):
- 4) Residuals vs Leverage: only a few high-leverage points.

For every +1000 sqm of GIA, staffing increases by about +2.3 FTE (roughly between +1.4 and +3.2, depending on uncertainty method). GIA explains ~48% of variation in staff numbers. Across all three models the relationship is consistent. The slope is strongly statistically significant in both the HC3 model and the robust model, meaning larger buildings reliably tend to have more staff.

The robust regression gives almost the same result as OLS, suggesting the fitted relationship isn't being driven by a small number of extreme observations.

```
[18]: # #-----
# # print residuals for analysis if needed
# #-----
# residuals = m1.resid
# fitted = m1.fittedvalues

# # indices of the top 3 absolute residuals
# # top3_idx = residuals.abs().sort_values(ascending=False).head(3).index
# sorted_residuals = residuals.abs().sort_values(ascending=False).head(3)
# # sorted_residuals
# top1_idx = residuals.abs().idxmax()
# data_ldn_exstg.loc[[top1_idx]]

# # print("Top 3 residual rows (largest abs residuals):")
# # data_ldn_exstg_regression_m1.loc[top3_idx]

[19]: # #-----
# #Plot regression line
# #-----
x, y = "b8_gia_1000_sqm", "b8_total_staff_fte"
cols = ["id", "lpa_name", "current_occupier"]
df = data_ldn_exstg_regression_m1
ht = "<br>".join([f"{c}: %{{customdata[{i}]}}" for i,c in enumerate(cols)]) +
    "<extra></extra>"
xl = np.linspace(df[x].min(), df[x].max(), 200)

fig = go.Figure([go.Scatter(x=df[x], y=df[y], mode="markers",
    marker=dict(size=7,opacity=0.9),
    name="Observations", customdata=df[cols].
    to_numpy(), hovertemplate=ht)])
for mdl, name, style in [(m1,"OLS",dict(color="black")), (m1_robust_se,"OLS_
    (HC3)",dict(color="grey",dash="dash")),
    (m1_robust_HuberT,"Robust (RLM)",dict(color="red"))]:
    a,b = mdl.params["const"], mdl.params[x]
    fig.add_trace(go.Scatter(x=xl, y=a+b*xl, mode="lines",
    line=dict(width=2,**style), name=name))

fig.update_layout(width=1100,height=600,template="plotly_white",
    title=dict(text="<br><span style='font-size:24px; font-weight:
    bold; color:%s;'>Baseline regression models' fitted lines</
    span>"%notes_colour,
    x=0.03,xanchor="left",y=0.98,yanchor="top",
    margin=dict(t=90),legend=dict(font=dict(color=labels_colour)))
```

```
fig.update_xaxes(title_text="Gros Internal Area,
↳1000sqm",title_font=dict(color=labels_colour),tickfont=dict(color=labels_colour))
fig.update_yaxes(title_text="Total
↳staff",title_font=dict(color=labels_colour),tickfont=dict(color=labels_colour))
fig.show()
```

```
[20]: # # -----
# # map residuals for analysis
# # -----
g = data_ldn_exstg_regression_m1.to_crs(4326).copy()
g["residual"] = m1.resid
g["lon"], g["lat"] = g.geometry.x, g.geometry.y

cols =
↳["id", "current_occupier", "lpa_name", "b8_gia_1000_sqm", "b8_total_staff_fte", "cluster_text"]
ht = "<br>".join([f"{c}: %{{customdata[{{i}}]}}" for i, c in enumerate(cols)]) +
↳"<extra></extra>"

fig = make_subplots(cols=2, horizontal_spacing=0.02,
↳subplot_titles=("Residuals (colour graded)", "All points
↳(black)"),
↳specs=[[{"type": "map"}, {"type": "map"}]])

fig.add_trace(go.Scattermap(
↳lat=g["lat"], lon=g["lon"], mode="markers",
↳marker=dict(size=9, color=g["residual"], colorscale="bluered", cmid=0,
↳opacity=1, showscale=True),
↳customdata=g[cols].to_numpy(), hovertemplate=ht, showlegend=False
), 1, 1)

fig.add_trace(go.Scattermap(
↳lat=g["lat"], lon=g["lon"], mode="markers",
↳marker=dict(size=9, color="black", opacity=1),
↳customdata=g[cols].to_numpy(), hovertemplate=ht, showlegend=False
), 1, 2)

fig.update_layout(
↳map=dict(style="carto-positron",
↳center=dict(lat=51.50758519898516, lon=-0.1281236104918927),
↳zoom=8),
↳map2=dict(style="carto-positron",
↳center=dict(lat=51.50758519898516, lon=-0.1281236104918927),
↳zoom=8),
↳height=650, width=1400,
↳title=dict(
↳text=(
```

```

        f"<span style='font-size:24px; font-weight:bold; color:
↪{notes_colour}'; '>"
        "Mapped residuals, m1</span><br>"
        f"<span style='font-size:13px; font-weight:300; color:
↪{notes_colour}; display:block; '>"
        "Red points: overestimated total_staff, blue: underestimated.</
↪span>"
    ),
    x=0.00, xanchor="left",
    y=0.95, yanchor="top"
),
legend=dict(font=dict(color=labels_colour)),
margin=dict(l=0, r=0, t=90, b=0)
)
fig.update_annotations(font=dict(size=14, color=labels_colour))
fig.show()

```

The residuals are positive on the north bank, and negative on the south bank of the River Thames. This might be a coincidence, or it might be related to differences in fiber network quality in different parts of London.

1.5 Linear Regression with Interaction Term

```

[21]: #_
↪
# Interaction Model: different intercepts + different slopes
# b8_total_staff_fte ~ b8_gia_1000_sqm * cluster_text
#_
↪
value_cols_m2 = ["b8_total_staff_fte", "b8_gia_1000_sqm", "cluster_text"]
treatment_ref = "Fringe"
# log_cols_m2 = ["b8_total_staff_fte", "b8_gia_1000_sqm"]
df_m2 = (data_ldn_exstg.dropna(subset=value_cols_m2).copy())
df_m2["cluster_text"] = df_m2["cluster_text"].astype("category")
#_
↪
# Drop outliers using IQR
for col in ["b8_total_staff_fte", "b8_gia_1000_sqm"]:
    q1, q3 = df_m2[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    df_m2 = df_m2[df_m2[col].between(low, high)]
#_
↪
# Interaction regression (different intercepts + different slopes)
m2 = smf.ols(

```

```

    f'b8_total_staff_fte ~ b8_gia_1000_sqm * C(cluster_text,
↳Treatment(reference="{treatment_ref}"))',
    data=df_m2
).fit()

# Interaction regression (different intercepts + different slopes)
m2_robust_se = smf.ols(
    f'b8_total_staff_fte ~ b8_gia_1000_sqm * C(cluster_text,
↳Treatment(reference="{treatment_ref}"))',
    data=df_m2
).fit(cov_type="HC3")

y, X = patsy.dmatrices(
    f'b8_total_staff_fte ~ b8_gia_1000_sqm * C(cluster_text,
↳Treatment(reference="{treatment_ref}"))',
    data=df_m2,
    return_type="dataframe"
)
m2_robust_HuberT = sm.RLM(y, X, M=sm.robust.norms.HuberT()).fit()

# # Same intercept
# m2_same_intercept = smf.ols(
#     'b8_total_staff_fte ~ b8_gia_1000_sqm + b8_gia_1000_sqm:C(cluster_text,
↳Treatment(reference="High density"))',
#     data=df_m2
# ).fit()

# m2_same_intercept = smf.ols(
#     'b8_total_staff_fte ~ b8_gia_1000_sqm_c + b8_gia_1000_sqm_c:
↳C(cluster_text, Treatment(reference="Medium density"))',
#     data=df_m2
# ).fit()

print(m2.summary())
print(m2_robust_se.summary())
print(m2_robust_HuberT.summary())

```

OLS Regression Results

```

=====
Dep. Variable:    b8_total_staff_fte    R-squared:                0.565
Model:            OLS                  Adj. R-squared:           0.483
Method:           Least Squares         F-statistic:              6.927
Date:             Tue, 25 Aug 2026       Prob (F-statistic):       0.00335
Time:             22:57:21              Log-Likelihood:           -89.916
No. Observations: 20                   AIC:                     187.8
Df Residuals:     16                   BIC:                     191.8
Df Model:         3

```

Covariance Type: nonrobust

```
=====
coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept
13.4375    10.386        1.294    0.214     -8.580     35.455
C(cluster_text, Treatment(reference="Fringe")) [T.Central]
15.6971    16.575        0.947    0.358    -19.440     50.834
b8_gia_1000_sqm
2.1194     0.671        3.158    0.006      0.697      3.542
b8_gia_1000_sqm:C(cluster_text, Treatment(reference="Fringe")) [T.Central]
0.5003     1.146        0.437    0.668     -1.929      2.930
=====
Omnibus:                1.742    Durbin-Watson:                1.490
Prob(Omnibus):           0.419    Jarque-Bera (JB):            0.972
Skew:                    0.060    Prob(JB):                    0.615
Kurtosis:                1.927    Cond. No.                    53.4
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Regression Results

```
=====
Dep. Variable:    b8_total_staff_fte    R-squared:                0.565
Model:            OLS                  Adj. R-squared:            0.483
Method:           Least Squares        F-statistic:              7.594
Date:             Tue, 25 Aug 2026      Prob (F-statistic):       0.00223
Time:             22:57:21              Log-Likelihood:          -89.916
No. Observations: 20                  AIC:                     187.8
Df Residuals:     16                  BIC:                     191.8
Df Model:          3
Covariance Type:  HC3
=====
```

```
=====
coef      std err          z      P>|z|      [0.025      0.975]
-----
Intercept
13.4375     8.994        1.494    0.135     -4.191     31.066
C(cluster_text, Treatment(reference="Fringe")) [T.Central]
15.6971    18.861        0.832    0.405    -21.270     52.664
b8_gia_1000_sqm
2.1194     0.577        3.671    0.000      0.988      3.251
b8_gia_1000_sqm:C(cluster_text, Treatment(reference="Fringe")) [T.Central]
0.5003     1.259        0.398    0.691     -1.966      2.967
=====
```

```
=====
Omnibus:                1.742    Durbin-Watson:                1.490
Prob(Omnibus):          0.419    Jarque-Bera (JB):        0.972
Skew:                   0.060    Prob(JB):                0.615
Kurtosis:               1.927    Cond. No.                53.4
=====
```

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

Robust linear Model Regression Results

```
=====
Dep. Variable:    b8_total_staff_fte    No. Observations:    20
Model:            RLM                  Df Residuals:        16
Method:           IRLS                 Df Model:            3
Norm:             HuberT
Scale Est.:       mad
Cov Type:         H1
Date:             Tue, 25 Aug 2026
Time:             22:57:21
No. Iterations:   2
=====
```

```
=====
coef      std err      z      P>|z|      [0.025      0.975]
-----
Intercept
13.4375    10.386      1.294    0.196    -6.918    33.794
C(cluster_text, Treatment(reference="Fringe"))[T.Central]
15.6971    16.575      0.947    0.344   -16.789    48.183
b8_gia_1000_sqm
2.1194     0.671      3.158    0.002     0.804     3.435
b8_gia_1000_sqm:C(cluster_text, Treatment(reference="Fringe"))[T.Central]
0.5003     1.146      0.437    0.662    -1.746     2.747
=====
```

If the model instance has been used for another fit with different fit parameters, then the fit options might not be the correct ones anymore .

Staffing increases strongly with building size: in the Central group, staff increases by ~2.6 FTE per additional 1,000 sqm. There is no statistically reliable evidence that Fringe sites have a different baseline staffing level or a different staffing-GIA slope, and the robust versions give the same conclusions, suggesting results are not driven by outliers or heteroskedasticity.

```
[22]: value_col_x, value_col_y, group_col = "b8_gia_1000_sqm", "b8_total_staff_fte", "
      ↪ "cluster_text"
df_plot = df_m2.copy()
baseline = treatment_ref    # must match your Treatment(reference="Central")
```



```

xline = np.linspace(df_plot[value_col_x].min(), df_plot[value_col_x].max(), 200)
groups_sorted = sorted(df_plot[group_col].dropna().unique())

fig = go.Figure()

for i, g in enumerate(groups_sorted):
    d = df_plot[df_plot[group_col] == g]
    col = palette[i % len(palette)]

    # points
    fig.add_trace(go.Scatter(
        x=d[value_col_x], y=d[value_col_y], mode="markers",
        marker=dict(size=7, color=col, opacity=0.9),
        name=str(g),
        customdata=d[cols_to_show].to_numpy(),
        hovertemplate=hovertemplate
    ))

    # # ----- OLS line params -----
    # a_ols = m2.params["Intercept"]
    # b_ols = m2.params[value_col_x]
    # if g != baseline:
    #     a_ols += m2.params.get(f'C(cluster_text,
    ↪Treatment(reference="{baseline}"))[T.{g}]', 0)
    #     b_ols += m2.params.get(f'{value_col_x}:C(cluster_text,
    ↪Treatment(reference="{baseline}"))[T.{g}]', 0)

    # fig.add_trace(go.Scatter(
    #     x=xline, y=a_ols + b_ols * xline, mode="lines",
    #     line=dict(color=col, width=2),
    #     name=f"{g} (OLS)", showlegend=False, hoverinfo="skip"
    # ))

    # ----- Robust (Huber) line params -----
    a_rlm = m2_robust_HuberT.params["Intercept"]
    b_rlm = m2_robust_HuberT.params[value_col_x]
    if g != baseline:
        a_rlm += m2_robust_HuberT.params.get(f'C(cluster_text,
    ↪Treatment(reference="{baseline}"))[T.{g}]', 0)
        b_rlm += m2_robust_HuberT.params.get(f'{value_col_x}:C(cluster_text,
    ↪Treatment(reference="{baseline}"))[T.{g}]', 0)

    fig.add_trace(go.Scatter(
        x=xline, y=a_rlm + b_rlm * xline, mode="lines",
        line=dict(color=col, width=2, dash="dot"),
        name=f"{g} (RLM)", showlegend=False, hoverinfo="skip"
    ))

```

```

))

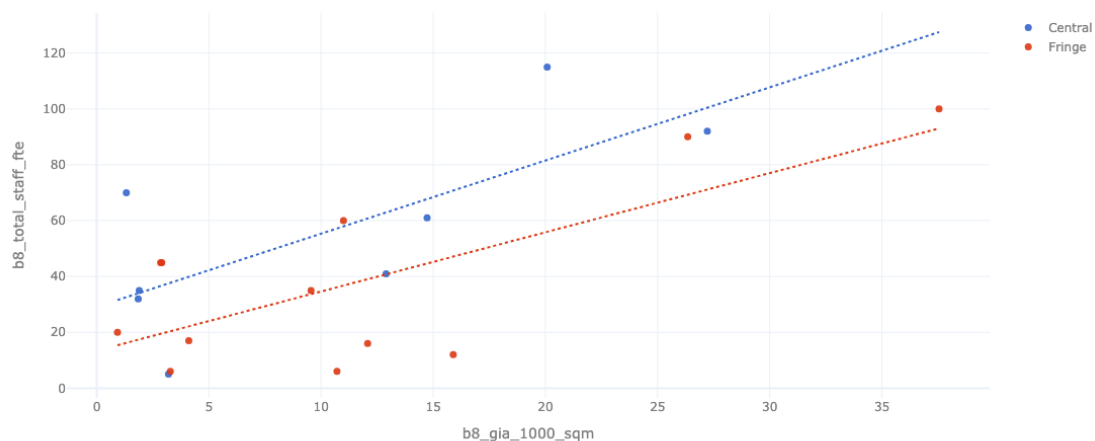
fig.update_layout(
    width=1100, height=600, template="plotly_white",
    title=dict(
        text=(
            f"<br><span style='font-size:24px; font-weight:bold; color:
↪{notes_colour}; display:block;'>"
            "Staff vs GIA (separate slopes by cluster)"
            "</span><br>"
            f"<span style='font-size:13px; font-weight:300; color:
↪{notes_colour}; display:block;'>"
            "Solid = OLS interaction model, dotted = Robust (Huber) interaction_
↪model."
            "</span><br>"
        ),
        x=0.03, xanchor="left", y=0.98, yanchor="top",
    ),
    margin=dict(t=140),
    legend=dict(font=dict(color=labels_colour)),
)

fig.update_xaxes(title_text=value_col_x, title_font=dict(color=labels_colour),
↪tickfont=dict(color=labels_colour))
fig.update_yaxes(title_text=value_col_y, title_font=dict(color=labels_colour),
↪tickfont=dict(color=labels_colour))
fig.show()

```

Staff vs GIA (separate slopes by cluster)

Solid = OLS interaction model, dotted = Robust (Huber) interaction model.



Compared with the earlier non-interaction models, the main message is consistent: **staffing increases as GIA increases**, at roughly **~2.2–2.6 FTE per additional 1,000 sqm** (simple model slope = 2.24; interaction model Central slope = 2.62). The interaction model improves explanatory power (R^2 rises from **0.46** to **0.57**), suggesting that allowing for cluster differences helps capture some extra variation, but the **Fringe intercept and Fringe×GIA interaction terms are not statistically significant**, so there is **no strong evidence** that Fringe sites follow a meaningfully different staffing–size relationship than Central sites in this sample. Taken together, both models imply that **GIA is the dominant driver of staffing**, while cluster effects appear secondary and may be difficult to estimate precisely given the small number of observations.

[]: